

React Compiler: Introduction

React Compiler kya hai?

React Compiler ek build-time tool hai jo automatically tumhari React application ko optimize karta hai. Build-time ka matlab hai ki optimization application ke build hone ke dauran hoti hai, browser mein runtime par nahi.

Ye:

- Normal JavaScript ke saath kaam karta hai
- Rules of React ko samajhta hai
- Components aur values ko automatically memoize karta hai
- Unnecessary re-renders reduce karta hai
- Manual useMemo, useCallback aur React.memo ki need ko kaafi kam kar deta hai

Isko use karne ke liye generally tumhe apne existing components ko kisi special style mein rewrite karne ki zarurat nahi hoti.

React Compiler ki zarurat kyon hai?

React applications often bina kisi optimization ke sufficiently fast hoti hain. Lekin large application mein unnecessary re-renders aur expensive calculations performance ko affect kar sakte hain.

Compiler ke bina developers manually ye tools use karte hain:

- React.memo()
- useMemo()
- useCallback()

Problem ye hai ki manual memoization:

- Tedious ho sakti hai
- Code ko complicated banati hai
- Easily incorrect ho sakti hai
- Additional code maintain karna padta hai
- Kabhi-kabhi optimization ke bawajood subtle bugs aa jaate hain

React Compiler ye mental burden developer se hata kar automatic optimization apply karta hai.

React Compiler ke pehle

Compiler ke bina code kuch is tarah manually memoize kiya ja sakta hai:

```
1  import { useMemo, useCallback, memo } from 'react';
2
3  const ExpensiveComponent = memo(function ExpensiveComponent({
4    data,
5    onClick
6  }) {
7    const processedData = useMemo(() => {
8      return expensiveProcessing(data);
9    }, [data]);
10
11    const handleClick = useCallback((item) => {
12      onClick(item.id);
13    }, [onClick]);
14
15    return (
16      <div>
17        {processedData.map(item => (
18          <Item
19            key={item.id}
20            onClick={() => handleClick(item)}
21          />
22        ))}
23      </div>
24    );
25  });
```

Yahan teen manual optimizations hain:

React.memo()

```
1  memo(function ExpensiveComponent() {
2    // ...
3  });
```

Ye component ke re-render ko skip karne ki koshish karta hai jab uske props change nahi hue hon.

useMemo()

```
1  const processedData = useMemo(() => {
2    return expensiveProcessing(data);
```

```
3 }, [data]);
```

Ye calculation ka result save karta hai. `expensiveProcessing(data)` dobara tabhi chalega jab data dependency change hogi.

`useCallback()`

```
1 const handleClick = useCallback((item) => {  
2   onClick(item.id);  
3 }, [onClick]);
```

Ye function reference ko stable rakhne ki koshish karta hai. Naya function tab create hoga jab `onClick` dependency change hogi.

Is manual memoization mein subtle bug kya hai?

Problem is line mein hai:

```
1 <Item  
2   key={item.id}  
3   onClick={() => handleClick(item)}  
4 />
```

Although `handleClick` ko `useCallback` mein wrap kiya gaya hai, lekin:

```
1 () => handleClick(item)
```

khud ek naya arrow function hai.

Component jab bhi render hoga, ye arrow function dobara create hoga. Isliye `Item` ko har render par ek naya `onClick` prop reference milega.

Conceptually:

```
1 const firstFunction = () => handleClick(item);  
2 const secondFunction = () => handleClick(item);  
3  
4 console.log(firstFunction === secondFunction);  
5 // false
```

Dono functions same kaam karte hain, lekin JavaScript mein unke references different hain. Is wajah se `Item` ki memoization break ho sakti hai aur wo unnecessary re-render kar sakta hai.

React Compiler arrow function ke saath ya uske bina bhi is case ko correctly optimize kar sakta hai.

React Compiler ke baad

Compiler enabled hone par tum simpler code likh sakte ho:

```
1  function ExpensiveComponent({ data, onClick }) {
2    const processedData = expensiveProcessing(data);
3
4    const handleClick = (item) => {
5      onClick(item.id);
6    };
7
8    return (
9      <div>
10        {processedData.map(item => (
11          <Item
12            key={item.id}
13            onClick={() => handleClick(item)}
14          />
15        ))}
16      </div>
17    );
18  }
```

Ab code mein manually:

- React.memo
- useMemo
- useCallback

use nahi kiye gaye.

React Compiler code ko analyze karke suitable memoization automatically apply karta hai, jisse application sirf necessary situations mein re-render hoti hai.

Iska major benefit ye hai ki developer optimization details handle karne ke bajay application features par focus kar sakta hai.

React Compiler kis type ki memoization karta hai?

React Compiler ka primary focus **update performance improve karna** hai. Yaani jab existing components re-render hote hain, tab unnecessary work ko skip karna.

Ye mainly do situations optimize karta hai.

1. Cascading re-renders skip karna

React mein parent component re-render hone par normally uske child components bhi re-render ho sakte hain, even agar unse related data change na hua ho.

Example:

```
1 function FriendList({ friends }) {
2   const onlineCount = useFriendOnlineCount();
3
4   if (friends.length === 0) {
5     return <NoFriends />;
6   }
7
8   return (
9     <div>
10      <span>{onlineCount} online</span>
11
12      {friends.map((friend) => (
13        <FriendListCard
14          key={friend.id}
15          friend={friend}
16        />
17      ))}
18
19      <MessageButton />
20    </div>
21  );
22 }
```

Suppose onlineCount change hota hai. Isse FriendList re-render hoga.

Normal React behavior mein uske children bhi re-render ho sakte hain:

```
1 <FriendListCard />
2 <MessageButton />
```

Lekin MessageButton ka onlineCount se koi direct relation nahi hai.

React Compiler identify kar sakta hai ki kaunsa JSX safely reuse ho sakta hai. Is example mein wo unnecessary JSX recreation aur MessageButton jaise unrelated component ke re-render ko avoid kar sakta hai.

Is tarah ki focused updating ko kabhi-kabhi **fine-grained reactivity** kaha jata hai.

2. Expensive calculations ko memoize karna

React Compiler component ya hook ke andar use hone wali expensive calculation ko automatically memoize kar sakta hai.

```
1 function expensivelyProcessAReallyLargeArrayOfObjects(items) {  
2   // Expensive processing  
3 }  
4  
5 function TableContainer({ items }) {  
6   const data =  
7     expensivelyProcessAReallyLargeArrayOfObjects(items);  
8  
9   // ...  
10 }
```

TableContainer ek React component hai, isliye compiler iske andar calculation ko memoize kar sakta hai.

Agar items relevant tarike se change nahi hote, compiler calculation ka previous result reuse kar sakta hai.

Compiler har JavaScript function ko memoize nahi karta

Ye distinction important hai:

```
1 function expensivelyProcessAReallyLargeArrayOfObjects(items) {  
2   // Ye normal function hai  
3 }
```

React Compiler normal standalone function ko directly globally memoize nahi karta. Wo primarily **React components aur hooks** ke andar optimization apply karta hai.

Iska matlab agar same expensive function multiple components mein call ho raha hai:

```
1 function ComponentA({ items }) {  
2   const data =  
3     expensivelyProcessAReallyLargeArrayOfObjects(items);  
4 }  
5  
6 function ComponentB({ items }) {  
7   const data =  
8     expensivelyProcessAReallyLargeArrayOfObjects(items);  
9 }
```

to memoization result automatically ComponentA aur ComponentB ke beech share nahi hoga. Same input hone ke baad bhi calculation alag components mein separately run ho sakti hai.

Agar function genuinely बहुत expensive hai aur multiple places par same input ke saath use ho raha hai, to React ke bahar us function ki own shared memoization consider ki ja sakti hai.

Lekin code ko complicated banane se pehle profiling karni chahiye, taaki confirm ho sake ki calculation actually expensive hai.

Kya React Compiler try karna chahiye?

Provided documentation React Compiler use karna start karne ke liye encourage karti hai.

Compiler abhi React ke liye optional addition hai, lekin future mein kuch React features ko completely work karne ke liye compiler ki zarurat ho sakti hai.

Kya production mein use karna safe hai?

Provided page ke according:

- React Compiler stable hai
- Production mein extensively test kiya gaya hai
- Meta jaisi companies mein production mein use hua hai

Lekin tumhare application mein rollout successful hoga ya nahi, ye do important cheezon par depend karta hai:

1. Tumhare codebase ki health
2. Code ne Rules of React ko kitni properly follow kiya hai

Compiler tumhare code ko analyze karta hai. Agar code Rules of React violate karta hai, to adoption aur compilation mein problems aa sakti hain.

Isliye existing project mein directly har jagah enable karne ke bajay incremental adoption aur testing useful hai.

Kaunse build tools supported hain?

React Compiler ko multiple build tools ke saath install kiya ja sakta hai, including:

- Babel
- Vite
- Metro
- Rsbuild

React Compiler ka current stable implementation primarily ek lightweight Babel plugin wrapper ke through available hai.

Core compiler ko Babel se tightly coupled nahi banaya gaya. React team SWC aur OXC teams ke saath first-class support par kaam kar rahi hai, taaki future mein compiler use karne ke liye build pipeline mein Babel dobara add karna necessary na ho.

Next.js support

Provided content ke according, Next.js users **version 15.3.1 aur uske baad ke versions** mein SWC-invoked React Compiler enable kar sakte hain.

useMemo, useCallback aur React.memo ka kya karein?

React Compiler enable karne ka matlab ye nahi hai ki ye APIs suddenly invalid ho gayi hain.

Compiler by default apni analysis aur heuristics ke basis par code memoize karta hai. Most cases mein compiler ki memoization manually written memoization jitni, ya usse zyada precise ho sakti hai.

Lekin kuch situations mein developer ko exact control chahiye hota hai.

New code ke liye

New code mein general recommendation hai:

- Memoization ke liye pehle compiler par rely karo
- Jab precise control genuinely required ho, tab useMemo ya useCallback use karo

Example situation: koi memoized value effect dependency ke roop mein use ho rahi ho aur tum ensure karna chahte ho ki effect repeatedly fire na kare jab dependencies meaningfully change nahi hui hain.

Existing code ke liye

Existing application mein agar pehle se:

```
1  useMemo()  
2  useCallback()  
3  React.memo()
```


use ho raha hai, to usko immediately remove karna necessary nahi hai.

Documentation do approaches suggest karti hai:

1. Existing memoization ko as-is chhod do
2. Remove karna ho to carefully test karo

Manual memoization remove karne se compiler ka output aur application behavior change ho sakta hai. Isliye mechanical cleanup, yaani bina testing ke sari memoization remove karna, recommended approach nahi hai.

Incremental adoption kya hota hai?

Page incremental adoption ki details nahi deta, lekin ise getting-started resources ke ek dedicated topic ke roop mein list karta hai.

Iska purpose existing codebase mein compiler ko gradually roll out karne ki strategies provide karna hai, instead of necessarily poori application ko ek hi baar compiler par shift karna.

Ye especially useful ho sakta hai jab:

- Existing application large ho
 - Manual memoization already extensively used ho
 - Rules of React violations ka doubt ho
 - Production rollout carefully test karna ho
-

Debugging aur troubleshooting

Compiler use karte waqt agar kuch wrong hota hai, documentation iske liye dedicated debugging and troubleshooting section provide karti hai.

Iska focus compiler-related issues identify aur fix karna hai. Introduction page detailed debugging procedure nahi deta, balki dedicated guide ki taraf point karta hai.

React libraries ke liye compiler

React Compiler sirf applications ke liye nahi hai. React library authors bhi compiled code ship kar sakte hain.

Provided introduction mein **Compiling Libraries** ko dedicated topic ke roop mein list kiya gaya hai, jisme compiled library code ship karne ki best practices cover ki jaati hain.

Simple mental model

React Compiler ko is tarah samjho:

Compiler ke bina

Developer ko manually batana pad sakta hai:

```
1 "Is calculation ko cache karo."  
2 "Is function ka reference stable rakho."  
3 "Is component ko unnecessarily render mat karo."
```

Iske liye developer likhta hai:

```
1 useMemo()  
2 useCallback()  
3 React.memo()
```

Compiler ke saath

Developer comparatively natural React code likhta hai:

```
1 function Component({ data }) {  
2   const processedData = processData(data);  
3  
4   return <UI data={processedData} />;  
5 }
```

Compiler code aur dependencies ko analyze karke automatically decide karta hai:

- Kya safely reuse ho sakta hai
 - Kaunsa calculation memoize ho sakta hai
 - Kaunsa JSX recreate karne ki zarurat nahi hai
 - Kaunsa child re-render skip kiya ja sakta hai
-

Important takeaways

1. **React Compiler ek build-time optimization tool hai.**
2. Ye plain JavaScript aur Rules of React ko understand karta hai.
3. Ye manual memoization ki need ko significantly reduce karta hai.
4. Ye cascading child re-renders avoid kar sakta hai.

5. Ye components aur hooks ke andar expensive calculations memoize kar sakta hai.
6. Ye standalone functions ke results ko globally memoize nahi karta.
7. Memoization different components ke beech automatically share nahi hoti.
8. Existing useMemo, useCallback aur React.memo ko immediately remove nahi karna chahiye.
9. New code mein compiler par rely karke manual memoization ko precise-control escape hatch ki tarah use kiya ja sakta hai.
10. Production rollout codebase health aur Rules of React compliance par depend karta hai.
11. Babel, Vite, Metro aur Rsbuild jaise build tools supported hain.
12. Next.js 15.3.1+ mein SWC-invoked React Compiler enable kiya ja sakta hai.

External links mentioned on the page

- [Open the initial React Compiler example in Playground](#)
- [Open the re-rendering example in React Compiler Playground](#)
- [Open the expensive-calculation example in React Compiler Playground](#)
- <https://github.com/oxc-project/oxc/issues/10048>
- <https://github.com/vercel/next.js/releases/tag/v15.3.1>
- <https://github.com/reactwg/react-compiler>